

Algorithme de parcours et d'exploration

→ Entrée un graphe quelconque

DFS parcours en profondeur

En distribué

BFS parcours en largeur

DFS + BFS en "centralisé"

DFS (G matrice d'adjacence)

$\begin{cases} 0 & \text{non adjacent} \\ 1 & \text{adjacent.} \end{cases}$

→ graphe quelconque simple sans boucles non connexes

marque[] * tableau boolean init à faux, indique les sommets déjà exploré
tant qu'il existe 1 sommets tq marque[s] faux.

| DFS(S, G, marque).

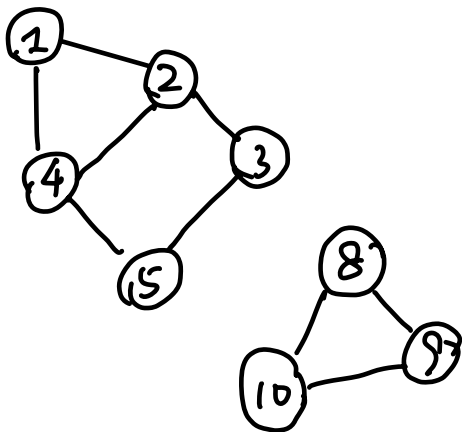
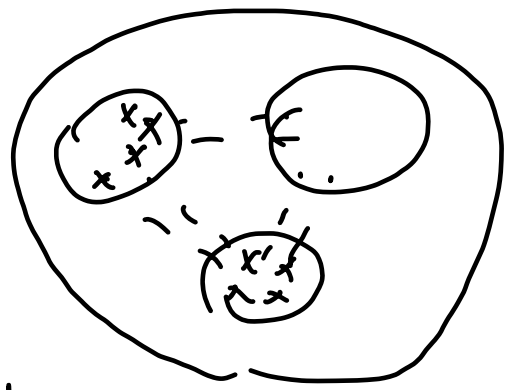
DFS(S, G, marque)

marque[S] ← 1

afficher(S)

tant que il existe 1 voisin x de S tq marque[x] == faux.

| DFS(G, x, marque)



(1, 2, 3, 5, 4, 8, 9, 10)

arêtes qui correspondent aux appels recursifs forment 1 arbre couvrant du graphe (forêt quant non connexe)

Def: arbre couvrant: sous-graphe de G qui est 1 arbre et qui contient tous les sommets

1^{er} algo: propagation d'information avec retour PIF

entrée : graphe quelconque

principe : 1 initiateur qui envoie des jetons à ses voisins .

- 1 non initiateur à la première réception de jeton, il définit son père
- Lorsque il a reçu 1 jeton de tous ses voisins , il transmet le jeton à son père.

PIF

Algorithm 11 Algorithme de PIF, local au site i

———— **Connaissance** ————

$vois_i$: ensemble des voisins du site i

———— **Variables** ————

$nbRecu_i$: nombre de messages reçus, initialisé à 0

$pere_i$: identifiant de site, initialisé à $ndef$

$init_i \in \{Vrai, Faux\}$ initialisé à Faux

———— **Algorithme** ————

Initialement $\rightarrow$ /* Sur un site exactement */

$init_i \leftarrow Vrai$

for all $j \in vois_i$ **do**

Envoyer $Jeton()$ à j

Sur réception de $Jeton()$ **de** $k \rightarrow$

$nbRecu_i ++$

if $init_i$ **then**

if $nbRecu_i = |vois_i|$ **then**

Décider ; STOP_GLOBAL

else

if $pere_i = ndef$ **then**

$pere_i \leftarrow k$

for all $j \in vois_i \setminus \{k\}$ **do**

Envoyer $Jeton()$ à j

if $nbRecu_i = |vois_i|$ **then**

Envoyer $Jeton()$ à $pere_i$

complexité message $2m = O(m)$

3 type de message :

- messages de diffusion
- message de retour vers le père
- message d'exploration.

3 Parcours par circulation de jeton

Hypothèses : Graphe quelconque avec un initiateur. Canaux de communication asynchrones et fiables.

Principe : Un initiateur crée un jeton qui circule dans le graphe. Lorsque i reçoit le jeton pour la première fois, il désigne comme son père l'émetteur du jeton. Lorsque i reçoit le jeton, pour la première fois ou non, il l'envoie à un de ses voisins privé de son père et à qui il n'a pas déjà envoyé le jeton. Si i a déjà envoyé le jeton à tous ses voisins privé de son père, il l'envoie alors à son père. Finalement, on obtient un arbre enraciné en l'initiateur. Cet arbre n'a aucune propriété particulière.

Algorithm 12 Algorithme de parcours, local au site i

——— **Connaissance** ———

$vois_i$: ensemble des voisins du site i

——— **Variables** ———

$pere_i$: identifiant de site, initialisé à $ndef$

$utilise_i$: tableau de booléen (indiqué par l'identifiant des voisins), initialisé à *Faux*

——— **Algorithme** ———

Initialement $\rightarrow$ /* Sur un site exactement */

$pere_i \leftarrow i$

Choisir $j_0 \in vois_i$

$utilise_i[j_0] \leftarrow Vrai$

Envoyer Jeton() à j_0

Sur réception de Jeton() de $j_0 \rightarrow$

if $pere_i = ndef$ **then**

$pere_i \leftarrow j_0$

if $\forall j \in vois_i, utilise_i[j] = Vrai$ **then**

Décider ; STOP_GLOBAL

else

if $\exists j \in vois_i, j \neq pere_i \wedge \neg utilise_i[j]$ **then**

Choisir $j \in vois_i \setminus \{pere_i\} \wedge \neg utilise_i[j]$

$utilise_i[j] \leftarrow Vrai$

Envoyer Jeton() à j

else

$utilise_i[pere_i] \leftarrow Vrai$

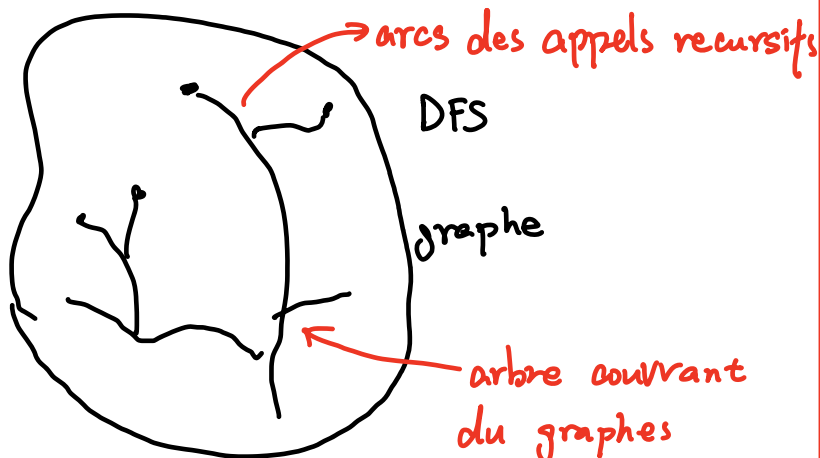
Envoyer Jeton() à $pere_i$

▷ Seul l'initiateur peut entrer dans ce "si"
n'est vrai que pour le site initiateur

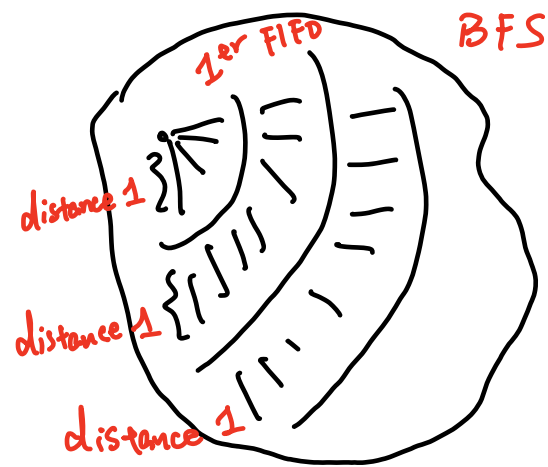
} est exécuté la dernière fois de tri l'échange

DFS : parcours en profondeur

BFS : parcours en largeur



arcs lient appels recursifs du
DFS forment un arbre couvrant



BFS
On explore les sommets par distance croissante au sommet d'origine

DFS en distribué

Il est basé sur l'algorithme de parcours par circulation de Jeton (avec le tableau).
En plus, on doit gérer spécifiquement à qui est envoyé le jeton pour "simuler" un parcours en profondeur.

En particulier, il faut préciser ce que se passe lorsque un nœud reçoit le jeton alors qu'il a déjà reçu précédemment.

Deux cas.

1. i n'a pas envoyé le jeton à j_0

2. i avait envoyé le jeton à j_0

1. On est en train d'explorer du côté de j_0 : Comme i avait déjà envoyé le jeton, il a fini son exploration, du coup renvoyer le jeton à j_0

↳ else



2^e cas exploration finis du côté de j_0 .

i doit envoyer le jeton à 1 autre voisin non exploré.